Rechnernetze und Netzwerkprogrammierung
Übungsblatt 01

Raphael Bleier, Devis, Fabio
Gruppe 1

Die sieben Programme liegen in diesem Ordner. Sie brauchen nur die Python-Standardbibliothek. Das Aufgabenblatt nennt Python 3; die Einführungsfolien empfehlen Version 3.8.5 oder neuer.

Getestet wurde mit Python 3.8.20.

```
uv run --python 3.8.20 python -m unittest discover -s tests -v
python threaded_calc.py zahlen.txt --threads 3
python thread_safety.py zahlen.txt --threads 4 --repeats 3
python producer_consumer.py --count 20 --queue-size 3
```

1.1 Python-Umgebung

hello.py gibt den Namen aus (standardmäßig Raphael Bleier, mit `--name` änderbar), die Python-Version über `platform.python_version()` und das aktuelle Arbeitsverzeichnis über `os.getcwd()`. Die virtuelle Umgebung hält die Python-Umgebung eines Projekts getrennt von anderen Installationen:

```
python3 -m venv .venv
source .venv/bin/activate
python hello.py
```

1.2 Text, Zeichenketten und Bytes

`str` enthält Unicode-Zeichen, `bytes` Bytewerte. UTF-8 braucht je nach Zeichen unterschiedlich viele Bytes. So ist `ä` ein Zeichen, in UTF-8 aber zwei Bytes (`c3 a4`). Zeichenanzahl und Bytelänge können deshalb verschieden sein.

Wird eine Bytefolge mit einer falschen Kodierung gelesen, kann das einen Dekodierfehler auslösen oder zu falschen Zeichen führen. Bei Netzwerkprogrammen ist die Unterscheidung wichtig, weil Bytes übertragen werden. Text wird vor dem Senden kodiert und nach dem Empfang mit der passenden Kodierung dekodiert.

1.3 Datei-I/O

`convert.py` bekommt Ein- und Ausgabedatei als Kommandozeilenparameter. Beim Lesen mit `newline=None` wandelt Python LF, CRLF und CR in ein einheitliches `\n` um. Das Programm schreibt anschließend LF, unabhängig vom Betriebssystem. `with` schließt die Dateien auch dann, wenn beim Lesen oder Schreiben ein Fehler auftritt.

Fehlende Parameter meldet `argparse`. Nicht vorhandene, nicht lesbare oder ungültig kodierte Eingaben sowie nicht beschreibbare Ausgaben führen zu einer Fehlermeldung. Ein- und Ausgabe dürfen nicht dieselbe Datei bezeichnen; der Vergleich erkennt auch Symlink- und Hardlink-Aliase, damit die Quelldatei nicht überschrieben wird.

1.4 Binäre Daten und Protokollnachrichten

Die ersten 32 Bit sind so aufgeteilt: Version in den Diagrammbits 0–4 (Wert 2), reservierte Bits in 5–13 (alle 0), D-Flag in 14, U-Flag in 15 und Sequenznummer in 16–31. Die Grafik zählt von links; im Big-Endian-Wort liegt das Versionsfeld daher in den höchstwertigen fünf Bits. Danach folgen die 32-Bit-Payload-Länge und die Payload. `struct`-Format `!II` kodiert beide Wörter in Network Byte Order.

`create_message` prüft Flags, Sequenznummer und Payload. `parse_message` prüft Version, reservierte Bits, positive Sequenznummer und ob die angegebene Payload-Länge genau zur Nachricht passt. Der Unit-Test vergleicht zusätzlich einen konkreten Byte-String und prüft fehlerhafte Eingaben.

Widerspruch in der Angabe. Die Bildprüfung bestätigt, dass die Round-Trip-Abbildung auf Seite 3 `40 00 00 2A 00 00 00 0B ...` zeigt. Der Feldkasten nennt keine Eingabewerte; aus den Bytes lassen sich aber D=U=0, Sequenznummer 42 und Payload-Länge 11 ablesen. Der Payload selbst ist nur als Auslassung dargestellt. Nach dem Bitfeldschema auf Seite 2 und der festen Vorgabe Version 2 ergibt das erste Byte `0x40` die Version 8. Für Version 2, D=U=0, Sequenznummer 42 und Payload-Länge 11 ergibt sich dagegen `10 00 00 2A 00 00 00 0B`. Die Angabe ist widersprüchlich; eine Klärung durch die Lehrveranstaltung steht aus. Die Implementierung folgt der ausdrücklich vorgegebenen Version 2 und dem Bitfeldschema.

1.5 Nebenläufigkeit mit Threads

Die Werte werden in zusammenhängende, überschneidungsfreie Bereiche aufgeteilt. Mit `divmod` bekommt jeder Thread zunächst gleich viele Elemente; die ersten Threads erhalten zusätzlich je ein Element, bis der Rest verteilt ist. So wird jeder Arraywert genau einmal verarbeitet, auch wenn die Länge nicht durch die Threadanzahl teilbar ist.

`Thread(...)` erzeugt ein Thread-Objekt. Erst `start()` startet dessen Funktion; `join()` wartet, bis sie fertig ist. Jeder Worker schreibt sein Teilergebnis in einen eigenen Eintrag der Ergebnisliste. So müssen die Threads die Summe nicht gleichzeitig ändern. Schlägt ein Threadstart fehl, wartet das Programm nur auf bereits gestartete Threads; Workerfehler werden nach dem Einsammeln weitergereicht.

Die Berechnung verwendet viele Quadratwurzeln als Beispiel für wiederholte Arbeit. Bei CPU-lastigem Python-Code beschleunigen Threads CPython wegen des Global Interpreter Lock nicht automatisch. Hier geht es um das Aufteilen der Arbeit und das Warten auf alle Worker.

1.6 Thread Safety

Thread Safety bedeutet, dass gleichzeitige Zugriffe keine falschen oder inkonsistenten Ergebnisse erzeugen. `result += value` ist als Lesen, Addieren und Schreiben zu verstehen. Wenn zwei Threads denselben alten Wert lesen, kann einer den Beitrag des anderen überschreiben.

Mit `zahlen.txt` ergaben drei direkte Läufe unter Python 3.8.20 jeweils 15088760.258000, genauso wie der Lock-Lauf und die sequenzielle Sollsumme. In diesem Versuch war kein verlorener Beitrag zu beobachten; das ist kein Beweis für Thread Safety, denn das Ergebnis hängt von Laufzeit und Thread-Scheduling ab. Die zusätzlichen Barriere-Kontrollläufe lagen bei etwa 3,77 Millionen und zeigten den verlorenen Update reproduzierbar. Der Kontrolllauf ersetzt den direkten Versuch nicht. Der Lock lässt immer nur einen Thread gleichzeitig auf die gemeinsame Summe zugreifen. Er bleibt nur während dieser kurzen Änderung aktiv, damit andere Threads nicht unnötig warten und die Arbeit nicht stärker als nötig serialisiert wird. Bei Workerfehlern wird die Barriere abgebrochen; Startfehler sammeln bereits gestartete Threads ein.

1.7 Producer-Consumer

`queue.Queue` ist für mehrere Threads geeignet, weil Einfügen und Entnehmen synchronisiert werden. Bei einer begrenzten Queue wartet `put()`, wenn sie voll ist; `get()` wartet, wenn sie leer ist. Ein Sentinel markiert das Ende der Produktion. `task_done()` meldet erledigte Einträge und `join()` wartet, bis die Queue abgearbeitet ist. Kurze Timeouts lassen die Worker auf Fehler reagieren; diese werden im aufrufenden Thread gemeldet. Bei Fehlern wird nicht auf unerledigte Einträge gewartet, sondern es werden die gestarteten Threads eingesammelt. Eine normale Liste bietet diese Synchronisation nicht.